PROJECT REPORT

on

Logic Level Synthesis of 2-bit ALU

Author

Vinit Kumar

Session: 2025–2026

Project Report

Logic Design and Digital Systems

Abstract

An ALU is one of the most important parts of any digital system. It does the arithmetic and logical operations required in processors and other devices. In this SEC project, we have designed a 2-bit ALU using Verilog and verified it by simulation. The ALU performs multiple operations, such as addition, subtraction, AND, OR, XOR, multiplication, and division by taking two 2-bit inputs with an opcode control signal. For division, the design provides the quotient, remainder, and handling of error for divide-by-zero conditions.

Our project follow a standard digital design flow. The coding is done in Verilog, and a testbench is used to verify simulation and waveform analysis using Icarus Verilog and GTKWave. Finally, all this is synthesised by Yosys.

In the next part, we analyze the efficiency in terms of delay, area, power, and the notable drawbacks of the Ripple Carry Adder (RCA) and the Carry Lookahead Adder (CLA). The results shows that CLA is faster, while RCA is simpler and uses less hardware. This highlights the important design compromise between speed and hardware cost in VLSI.

List of Figures

Table of Contents

Chapter 1 Introduction

1.1 Background

An Arithmetic Logic Unit (ALU) is an core component of all digital system. It carries out the arithmetic and logical operations needed by processors, microcontrollers, and embedded platforms. Contemporary computing systems rely on the ALU to handle data processing and instruction execution efficiently. As digital systems advance in both speed and complexity, designing high-performance ALUs has become a key focus area in digital electronics and VLSI engineering.

With the increasing demand for high-performance and low-power systems, designers must carefully balance speed, hardware area, and power consumption. Different adder architectures and logic implementations directly affect the performance of an ALU.

1.2 Theoretical Framework: ALU Operations and Adder Design

The ALU performs both arithmetic and logical functions. These operations include addition, subtraction, multiplication, and division, while logical operations include AND, OR, and XOR. The operation is selected using a control signal known as an opcode. Based on the opcode value, the ALU activates the required hardware path and generates the corresponding output.

Among all arithmetic operations, addition is one of the most frequently used operations, and it play a big role in determining ALU speed. The performance of the addition block depends on the type of adder used. In this project, two adder architectures are studied: Ripple Carry Adder (RCA) and Carry Lookahead Adder (CLA). RCA passes carry sequentially from one stage to the next, making it simple but slower. CLA reduces delay by generating carry signals in parallel, resulting in faster operation at the cost of higher hardware complexity.

1.2.1 Technical Challenges and Project Scope

ALU designing involves more than writing functional code. The design must be verified for all possible input combinations and analysed for timing, area, and efficiency. The operations such as multiplication and division require more logic resources compared to basic logical operations. Similarly, the choice of adder architecture affects propagation delay and overall performance of the system.

This project focuses on the complete digital design flow of a 2-bit ALU using Verilog HDL. It includes RTL modelling, simulation using Icarus Verilog, waveform verification using GTKWave, synthesis using Yosys and structural analysis of the generated hard-

ware. The project also compares RCA-based and CLA-based ALU implementations to understand the trade-off between speed and hardware cost.

1.2.2 Project Objective

The main objective of this project is to design and verify a functional 2-bit ALU capable of performing multiple arithmetic and logical operations. Another key objective is to compare two different adder architectures, the Ripple Carry Adder and Carry Lookahead Adder, within the ALU design.

The project aims to evaluate both implementations in terms of delay, area, power trend, and design complexity. Through this comparison, the study demonstrates how architectural changes can improve performance and highlights the practical considerations involved in digital and VLSI system design.

In a broader perspective, this work lays the foundation for designing more advanced ALUs used in modern processors, high-speed computing systems, and low-power embedded devices. The concepts explored in this project can be extended to higher-bit-width architectures and optimized designs, contributing to the development of efficient computing hardware for future technologies such as AI accelerators, edge computing systems, and next-generation integrated circuits.

Chapter 2 Introduction to Verilog

2.1 Verilog (Basic Building Block)

Verilog is a Hardware Description Language (HDL) that enables engineer to model and simulate digital hardware before physical implementation. It is a language that helps in describing hardware using code before actually building it on a proper chip or FPGA. Just like the C language is used to write software programs, Verilog is used to write hardware designs. It is widely used in digital electronics, VLSI design, processors, and embedded systems.

The most basic building block in Verilog is called a **module**. A module is a small hardware block with inputs, outputs, and logic in it. You can thinks of it like a black box: signals enter through the input pins, some operation happens inside, and the result comes out through the output pins. A simple gate, an adder, or even a full processor can all be written as modules.

Every module generally contains:

- **Inputs** – Signals coming into the circuit
- **Outputs** – Signals going out of the circuit
- **Internal signals** – Used for temporary connections inside the module
- **Logic** – The operation performed by the circuit

A simple Verilog module looks like this:

```
module example(input A, input B, output Y);
assign Y = A & B;
endmodule
```

In this example, two inputs A and B are given to the modules and output Y becomes the AND of both inputs.

Verilog also uses common data types such as:

- **wire** – Used for connections between components
- **reg** – Used to store values inside procedural blocks

So, whenever we design any digital circuit in Verilog, we first create modules and then connect them together to form larger systems.

2.2 Modelling (Behavioural / Structural)

In Verilog, there are many ways to describe a circuit. This is called **modelling style**. The two most common styles are:

1. Behavioural Modelling
2. Structural Modelling

These styles help in choosing whether we want to describe the function of the circuit or the actual hardware connections.

Behavioural Modelling

Behavioural modelling means we tell what the circuit should do. We does not worry about which gates are used inside. We write the required operation using simple statements like if, case, or arithmetic expressions.

For example:

```
result = A + B;
```

This means the output is the sum of A and B. We are not writing XOR gates or AND gates manually; instead, the synthesis tool decides how to create the adder hardware.

This style is very useful because:

- Code is short and easy to understand
- Design becomes faster
- Good for complex circuits

In this project, the ALU is mainly written using behavioural modelling.

Structural Modelling

Structural modelling describes how the circuit is physically implemented. We connect smaller blocks or logic gates to create a larger system.

For example, instead of writing:

```
A + B
```

we may connect full adders one by one to build an adder circuit.

This style is useful because:

- Gives better control over hardware design
- Helps understand circuit structure
- Useful for learning gate-level implementation

But for large designs, it becomes lengthy and harder to manage.

Simple Difference

- **Behavioural Modelling** = What the circuit does
- **Structural Modelling** = How the circuit is built

Both are important. Behavioural modelling is easier for coding, while structural modelling helps us understand the real hardware generated after synthesis.

2.3 Software

The digital design flow in this project utilizes a set of open-source Electronic Design Automation (EDA) tools to implement an complete RTL-to-GDSII process. Tools like Icarus Verilog and GTKWave are used for design simulation and waveform analysis, while Yosys performs logic synthesis and technology mapping. OpenLane, along with OpenROAD, automates the physical design stages, including floorplanning, placement, and routing. Finally, Magic VLSI, Netgen, and KLayout are used for verification and layout visualization, ensuring the correctness and integrity of the final chip design.

Table 2.1: RTL to GDSII Design Flow

Step	**Stage**	**Tool Used**	**Function (What it Does)**	**Output**
1	RTL Design	Icarus Verilog	Write and simulate Verilog code to verify functionality.	RTL file (.v)
2	Waveform Analysis	GTKWave	Visualize simulation results as waveforms.	VCD file (.vcd)
3	Logic Synthesis	Yosys	Convert RTL code into a gate-level netlist using logic gates.	Gate-level netlist
4	Technology Mapping	Yosys + Sky130 Std. Cell Library	Map generic gates to real CMOS standard cells.	Tech-mapped netlist
5	Flow Automation	OpenLane	Automate the complete RTL-to-GDS flow.	Full design flow
6	Floorplanning	OpenROAD	Define chip area, place I/O pins, and create base layout.	Floorplanned layout
7	Power Planning (PDN)	OpenROAD	Design VDD/GND power network (skipped for small design).	Power network
8	Placement	OpenROAD	Place standard cells optimally on the chip.	Placed design
9	Clock Tree Synthesis	OpenROAD	Distribute clock signal across the design (skipped here).	Clock network
10	Routing	OpenROAD	Create metal connections between cells.	Routed layout
11	DRC Check	Magic VLSI	Check layout for design rule violations.	DRC report
12	LVS Check	Netgen	Verify layout matches schematic/netlist.	LVS report
13	GDS Generation	OpenLane	Convert final layout to GDSII format.	GDS file (.gds)
14	Layout Visualization	KLayout	Visually inspect the final layout.	Visual layout

Chapter 3 Fundamentals of ALU

An Arithmetic Logic Unit (ALU) is a core component of any digital system, especially in processors and embedded systems. It is responsible for performing all the basic operations on data, which are required for computation and decision-making. The ALU works on binary inputs (0s and 1s) and produces a binary output based on the selected operation.

The term ALU is divided into three parts: Arithmetic (A), Logic (L), and Unit (U). The Arithmetic part deals with mathematical calculations, the Logic part handles logical comparisons, and the Unit represents the integration of both into one system. The ALU receives input data along with control signals, which decide which operation needs to be performed.

For example, if the control signal indicates addition, the ALU will perform addition on the inputs. If it indicates a logical operation like AND or OR, the ALU will perform that instead. This flexibility makes the ALU a very powerful building block. Understanding the ALU at a block level helps beginners see how simple circuits combine to perform complex tasks.

A modern ALU is a **combinational digital circuit** and one of the most important parts of a CPU. It performs arithmetic and logical operations directly on binary data.

Inputs of ALU:

- Operand A (2-bit)
- Operand B (2-bit)
- Opcode / Control Signal (operation selector)

Outputs of ALU:

- Result (4-bit)
- Remainder (for division)
- Status signals (Valid, Error)

Operations Performed: Addition, Subtraction, AND, OR, XOR, Multiplication, Division, and Modulus.

3.1 Block-Level Explanation of Arithmetic Unit (A)

The Arithmetic Unit is the part of the ALU that performs all mathematical operations on binary numbers. At the basic level, it mainly consists of adders, such as half adders and full adders. In most practical designs, multiple full adders are connected together to form a ripple carry adder, which can handle multi-bit addition.

For subtraction, a separate subtractor is usually not required. Instead, the concept of 2's complement is used. In this method, the number to be subtracted is first inverted and then 1 is added to it. This modified number is then added using the same adder circuit, making the design simple and efficient.

The Arithmetic Unit can also perform operations like increment (adding 1) and decrement (subtracting 1). These operations are controlled using multiplexers, which select the required input combination. The use of shared hardware reduces complexity and area.

Overall, the Arithmetic Unit focuses on numerical computation and forms the backbone of calculations inside the ALU.

3.2 Block-Level Explanation of Logic Unit (L)

The Logic Unit is responsible for performing logical operations on binary data. Unlike the Arithmetic Unit, it does not deal with numerical values but instead works on bitwise relationship between inputs. At the block level, it consists of basic logic gates such as AND, OR, NOT, NAND, NOR, and XOR.

Each gate performs a specific logical function. For example, the AND gate produces an output of 1 only when both inputs are 1, while the OR gate produces 1 if at least one input is 1. The NOT gate simply inverts the input, and the XOR gate gives 1 when the inputs are different.

In a typical design, outputs from all these gates are generated in parallel. A multiplexer is then used to select the desired logical operation based on control signals. This makes the Logic Unit flexible and efficient.

The Logic Unit is mainly used in decision-making processes, comparisons, and control operations in digital systems.

3.3 Block-Level Explanation of Unit (U)

The Unit (U) represents the complete ALU system where both the Arithmetic and Logic Units are combined into a single functional block. At the block level, the outputs from the Arithmetic Unit and Logic Unit are connected to a multiplexer.

This multiplexer is controlled by select lines, which decide whether the final output should come from the arithmetic operations or logical operations. This allows the ALU to perform multiple types of operations using a single integrated structure.

In addition to the main output, the ALU also generates status flags. These flags provide extra information about the result. For example, the Zero flag indicates that the output is zero, the Carry flag indicates a carry out from addition, and the Overflow flag indicates an error in signed arithmetic.

The integration of all these components makes the ALU highly efficient and versatile. It act as the decision-making and computation center of a processor, enabling it to execute instructions effectively.

Chapter 4 Implementation

The implementation of our project follows a simple and structured design flow.

First, we begin with **RTL Design**, where the circuit is written using Verilog. This step defines how the system should behave.

Next, we create a **Testbench**. This is used to apply different input values to the design and check whether the output is correct.

After that, we perform **Simulation** using Icarus Verilog. This step helps us verify the functionality of the design before implementing it in hardware.

The results of simulation are then viewed using **GTKWave**. It displays waveforms that show how signals change with time.

Once the design is verified, we move to **Synthesis** using Yosys. In this step, the Verilog code is converted into a gate-level netlist.

This produces a **Gate-Level Schematic** that represents the circuit's actual hardware structure.

Finally, we perform a **Performance Analysis** that compares different designs, such as the Ripple-Carry Adder (RCA) and the Carry-Lookahead Adder (CLA), in terms of delay and hardware usage.

Thus, the complete flow moves from design and testing to hardware generation and performance comparison.

4.1 Verilog Code Implementation

In this project, a 2-bit ALU is designed using Verilog HDL to perform both arithmetic and logical operations. The ALU takes two 2-bit inputs (A and B) and a 3-bit opcode to select the desired operation. Based on the opcode, operations such as addition, subtraction, AND, OR, XOR, multiplication, and division is performed.

Special attention is given to division, where a divide-by-zero condition is handled using `valid` and `error` signals. A testbench is also written to verify all operations using simulation. Additionally, an optimized version of the ALU using Carry Lookahead Adder (CLA) is implemented to improve addition speed.

Basic 2-bit ALU Verilog Code

```
module alu_2bit (
    input  [1:0] A,
```

```
    input  [1:0] B,
    input  [2:0] opcode,
    output reg [3:0] result,
    output reg valid,
    output reg error
);

always @(*) begin
    // default values
    result = 4'b0000;
    valid  = 1'b1;
    error  = 1'b0;

    case(opcode)

        // Addition
        3'b000: result = A + B;

        // Subtraction
        3'b001: result = A - B;

        // AND
        3'b010: result = A & B;

        // OR
        3'b011: result = A | B;

        // XOR
        3'b100: result = A ^ B;

        // Multiplication
        3'b101: result = A * B;

        // Division
        3'b110: begin
            if (B == 2'b00) begin
                result = 4'b0000;
                valid  = 1'b0;
                error  = 1'b1;
            end
            else begin
                result = A / B;
                valid  = 1'b1;
                error  = 1'b0;
            end
        end

        // Default case
        default: begin
            result = 4'b0000;
```

```
            valid  = 1'b0;
            error  = 1'b0;
        end

    endcase
end

endmodule
```

4.2 Testbench for RCA-based ALU

A testbench is a special piece of Verilog code used to verify whether a design is working correctly or no. It does not represent hardware; instead, it is used only for simulation purposes. In this project, the testbench is used to apply different input combinations to the ALU and observe the outputs.

The testbench generates input signals such as A, B, and opcode, and connects them to the ALU (called the Device Under Test or DUT). It also includes time delays to allow each operation to execute properly. Using commands like `$dumpfile` and `$dumpvars`, simulation results are stored in a waveform file, which can later be viewed in tools like GTKWave.

Different test cases are applied to verify all ALU operations, including addition, subtraction, logical operations, multiplication, and division. Special cases such as division by zero are also tested to ensure that the `valid` and `error` signals behave correctly. This helps in confirming that the design is functionally correct before moving to further stages.

RCA ALU Testbench Code

```
`timescale 1ns/1ps

module tb_alu;

reg [1:0] A, B;
reg [2:0] opcode;

wire [3:0] result;
wire valid;
wire error;

// Device Under Test (DUT)
alu_2bit uut (
    .A(A),
    .B(B),
    .opcode(opcode),
    .result(result),
    .valid(valid),
    .error(error)
);
```

```
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_alu);

    // Addition (3 + 3 = 6)
    A = 2'b11; B = 2'b11; opcode = 3'b000; #10;

    // Subtraction
    A = 2'b10; B = 2'b01; opcode = 3'b001; #10;

    // AND
    A = 2'b11; B = 2'b01; opcode = 3'b010; #10;

    // OR
    A = 2'b10; B = 2'b01; opcode = 3'b011; #10;

    // XOR
    A = 2'b11; B = 2'b01; opcode = 3'b100; #10;

    // Multiplication
    A = 2'b11; B = 2'b11; opcode = 3'b101; #10;

    // Division (valid case)
    A = 2'b10; B = 2'b01; opcode = 3'b110; #10;

    // Division by zero (error case)
    A = 2'b10; B = 2'b00; opcode = 3'b110; #10;

    $finish;
end

endmodule
```

4.3 Waveform Analysis

Waveform analysis is like watching the circuit think step by step. Instead of only seeing the final answer, we can see how every signal changes with time. This helps us check whether the ALU is working correctly.

In our project, waveform analysis is done using **GTKWave**. It reads the simulation output file and shows all signals in the form of timing waves.

The important signals shown in the waveform are:

- **A[1:0]** – First 2-bit input
- **B[1:0]** – Second 2-bit input
- **opcode[2:0]** – Selects which operation to perform

- **result[3:0]** – Main output of ALU
- **remainder[1:0]** – Used during division
- **valid** – Shows output is correct
- **error** – Shows divide-by-zero or invalid case

As time moves from left to right, the opcode changes. Each new opcode tells the ALU to do a different task such as addition, subtraction, AND, OR, XOR, multiplication, or division.

We can then compare the output result with the expected answer. If the waveform matches the correct values ,it means our design is working properly.

So, waveform analysis is a very useful step because it helps us find mistakes early and understand the behaviour of the circuit in a simple visual way.

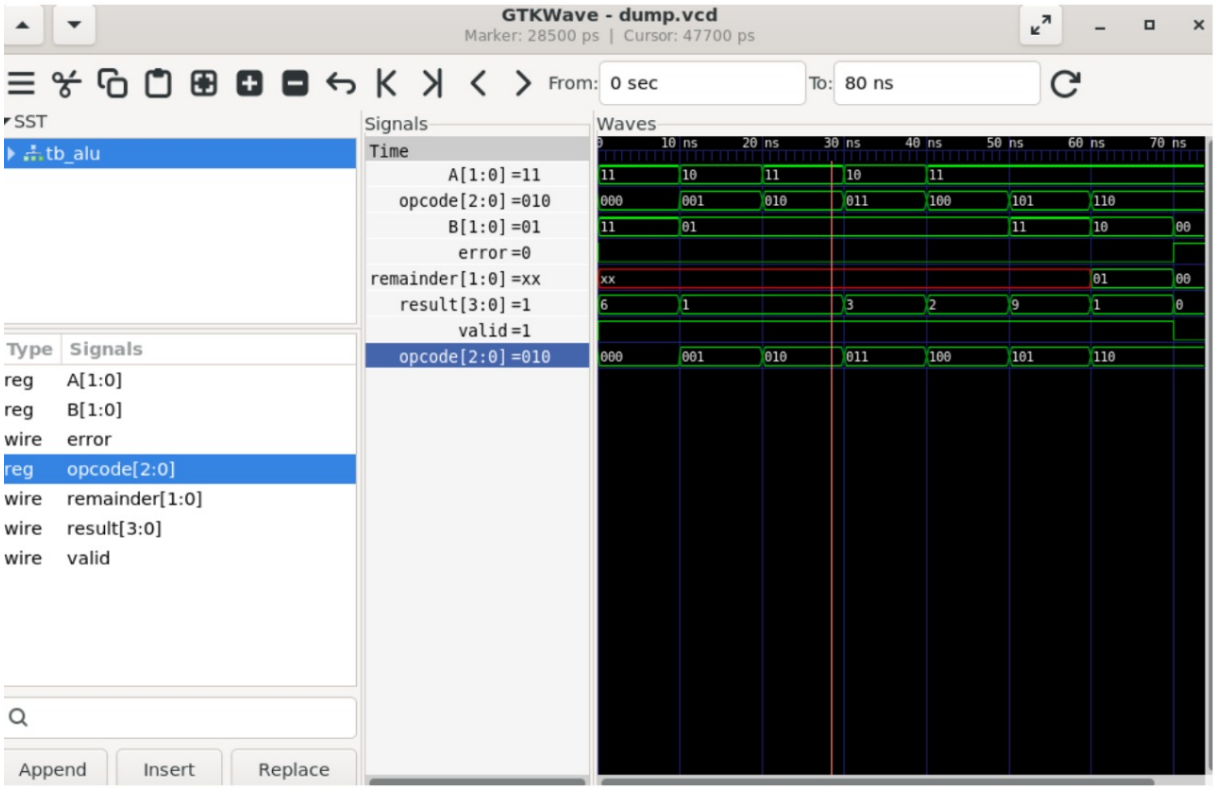

Figure 4.1: GTKWave simulation waveform of 2-bit ALU

4.4 Graphical User Interface (GUI) for Simulation and Visualization

A Graphical User Interface (GUI) helps users interact with tools in a visual and user-friendly way instead of using only command-line instructions. In this project, GUI-based tools are used to observe and understand the behavior of the ALU design more clearly.

After running the simulation using the testbench, the output is stored in a waveform file (with `.vcd` extension). This file can be opened using a GUI tool like GTKWave. In GTKWave, signals such as inputs (A, B, opcode) and outputs (result, valid, error) are displayed as waveforms over time. This makes it easy to see how the ALU responds to different inputs step by step.

The GUI allows users to zoom in ,zoom out, and analyze signal transitions in detail. For example, when the opcode changes, the corresponding change in output can be visually verified. It also helps in debugging, as incorrect behavior can be easily spotted by observing mismatches in expected and actual outputs.

In later stages of the design flow, GUI tools like KLayout are used to visualize the physical layout of the chip. This helps in understanding how the digital design is converted into an actual silicon layout. Overall, GUI tools make the design process more interactive, intuitive, and easier to understand, especially for beginners.

4.5 Block-Level Schematic

A block-level schematic is a simple diagram that shows how the main parts of the circuit are connected together. It helps us understand the hardware structure of the design without looking at long lines of code .

After writing the Verilog code and running synthesis, the design is converted into logic blocks and connections. This generated view is called the block-level schematic.

In our 2-bit ALU, the schematic mainly contains input lines, control logic, arithmetic blocks, logic gates, multiplexers, and output lines. Each block has job in producing the final result.

The two 2-bit inputs enter the circuit first. Then the opcode signal selects which operation must be performed. Based on this selection, the required block becomes active.

For example, if the opcode is for addition, the adder path is used. If the opcode is for AND or OR, the logic gate path is selected. A multiplexer then sends the correct output to the result line.

This schematic is handy because it shows how our Verilog code becomes real hardware. It also helps us check on design flow, understand connections, and study optimisation after synthesis.

4.6 Performance Analysis

4.6.1 (a) Delay Analysis

Delay analysis determines how long the ALU takes to produce its output after the inputs are applied. In digital circuits, this delay mainly comes from logic gates and the path through which signals travel. The total delay depends on how many gates are connected in series and how complex the operation is.

In the basic ALU design (RCA-based), the addition operation uses a ripple carry mechanism. In this method, the carry generated from one bit has to propagate to the next bit. Because of this sequential propagation, the delay increases with the number of

bits. Even in a 2-bit design, this effect can be observed, and for larger designs, it becomes more significant.

Other operations like AND, OR, and XOR have very small delays because they use simple logic gates. Multiplication and division take relatively more time as they involve more complex operations internally.

To improve delay performance, a Carry Lookahead Adder (CLA) is used in the optimized design. CLA reduces delay by calculating carry signals in advance instead of waiting for them to propagate. This makes addition faster compared to the ripple carry approach.

4.6.2 (b) Area Analysis

Area analysis refers to the amount of hardware resources required to implement the ALU on a chip. It mainly depends on the number of logic gates, wires, and standard cells used in the design. A design with fewer components occupies less silicon area and is generally more efficient.

The basic ALU design uses simple structures like ripple carry adders and logic gates, which require relatively less area. Since the design is small (2-bit), the overall area is also minimal. However, as the number of operations increases, additional hardware such as multiplexers and control logic is required, which increases the area.

In the CLA-based ALU, additional logic is needed to compute generate and propagate signals, as well as carry lookahead equations. This increases the hardware complexity and hence the area compared to the basic RCA design.

Therefore, there is always a trade-off between delay and area. The RCA design is simpler and uses less area but is slower, while the CLA design is faster but requires more area. Designers choose the appropriate approach based on system requirements.

Chapter 5 Implementation of CLA Adder in ALU

A Carry Lookahead Adder (CLA) is an improved type of adder used to reduce the delay caused by carry propagation. In a Ripple Carry Adder (RCA), each carry waits for the previous carry, which makes the operation slower. In CLA, the carry signals are calculated in advance using the Generate and Propagate logic.

Because of this smart carry calculation, CLA gives faster addition results compared to RCA. This makes it very useful in high-speed digital systems, processors, and ALUs.

In this project, the normal addition block of the ALU is replaced with a 2-bit Carry Lookahead Adder, while all other operations, such as subtraction, logic operations, multiplication, and division, remain the same.

5.1 CLA RTL Code

The addition path of the ALU is rebuilt around a 2-bit Carry Lookahead Adder. Instead of letting the carry ripple from one bit to the next, the Generate (G) and Propagate (P) signals for each bit are computed first, and the carries are derived directly from them:

$\mathrm{G}_0 = A_0 B_0, \quad G_1 = A_1 B_1$

$P_0 = A_0 \oplus B_0, \quad P_1 = A_1 \oplus B_1$

$C_1 = G_0 + P_0 C_0, \quad C_2 = G_1 + P_1 C_1$

$S_0 = P_0 \oplus C_0, \quad S_1 = P_1 \oplus C_1$

Subtraction reuses the same CLA adder via 2's complement ($A - B = A + \overline{B} + 1$), and only the 2-bit sum is kept as the result (the adder's carry-out in this case indicates borrow/no-borrow, not part of the magnitude). AND, OR, XOR, multiplication, and division are unchanged from `alu_2bit`, with a `remainder` output added for division.

alu_cla.v – CLA-based ALU

```
module alu_cla (
    input  [1:0] A,
    input  [1:0] B,
    input  [2:0] opcode,
    output reg [3:0] result,
    output reg [1:0] remainder,
    output reg       valid,
    output reg       error
);

    // 2-bit Carry Lookahead Adder core.
    // Generate (G) and Propagate (P) signals are computed per bit, and
```

```
    // the carries are derived directly from G/P instead of rippling.
    function [2:0] cla_add;
        input [1:0] x;
        input [1:0] y;
        input       cin;
        reg         g0, g1;   // generate
        reg         p0, p1;   // propagate
        reg         c0, c1, c2;
        reg         s0, s1;
        begin
            g0 = x[0] & y[0];
            g1 = x[1] & y[1];
            p0 = x[0] ^ y[0];
            p1 = x[1] ^ y[1];

            c0 = cin;
            c1 = g0 | (p0 & c0);
            c2 = g1 | (p1 & c1);

            s0 = p0 ^ c0;
            s1 = p1 ^ c1;

            cla_add = {c2, s1, s0}; // {carry-out, sum[1:0]}
        end
    endfunction

    reg [2:0] add_res;
    reg [2:0] sub_res;

    always @ (*) begin
        // default values
        result    = 4'b0000;
        remainder = 2'b00;
        valid     = 1'b1;
        error     = 1'b0;

        case (opcode)
            // Addition (CLA)
            3'b000: begin
                add_res = cla_add(A, B, 1'b0);
                result  = {1'b0, add_res};
            end
            // Subtraction (A - B via 2's complement, using the same
            // CLA adder). The adder's carry-out here is a borrow/
            // no-borrow flag, not part of the numeric magnitude, so
            // only the 2-bit sum is kept.
            3'b001: begin
                sub_res = cla_add(A, ~B, 1'b1);
                result  = {2'b00, sub_res[1:0]};
            end
```

```
            // AND
            3'b010: result = A & B;
            // OR
            3'b011: result = A | B;
            // XOR
            3'b100: result = A ^ B;
            // Multiplication
            3'b101: result = A * B;
            // Division
            3'b110: begin
                if (B == 2'b00) begin
                    result    = 4'b0000;
                    remainder = 2'b00;
                    valid     = 1'b0;
                    error     = 1'b1;
                end else begin
                    result    = A / B;
                    remainder = A % B;
                    valid     = 1'b1;
                    error     = 1'b0;
                end
            end
            // Default case
            default: begin
                result    = 4'b0000;
                remainder = 2'b00;
                valid     = 1'b0;
                error     = 1'b0;
            end
        endcase
    end

endmodule
```

This module was verified in simulation against the CLA testbench in Section 5.2, confirming correct results for all eight operations, including the divide-by-zero error path.

5.2 CLA Testbench Code

The following testbench is used to verify the working of the ALU with the Carry Lookahead Adder implementation.

CLA ALU Testbench Code

```
`timescale 1ns/1ps

module tb_alu_cla;
```

```
reg [1:0] A, B;
reg [2:0] opcode;

wire [3:0] result;
wire [1:0] remainder;
wire valid;
wire error;

// Device Under Test (DUT)
alu_cla uut (
    .A(A),
    .B(B),
    .opcode(opcode),
    .result(result),
    .remainder(remainder),
    .valid(valid),
    .error(error)
);

initial begin
    $dumpfile("cla_dump.vcd");
    $dumpvars(0, tb_alu_cla);

    // Addition (3 + 3 = 6)
    A = 2'b11; B = 2'b11; opcode = 3'b000; #10;

    // Subtraction
    A = 2'b10; B = 2'b01; opcode = 3'b001; #10;

    // AND
    A = 2'b11; B = 2'b01; opcode = 3'b010; #10;

    // OR
    A = 2'b10; B = 2'b01; opcode = 3'b011; #10;

    // XOR
    A = 2'b11; B = 2'b01; opcode = 3'b100; #10;

    // Multiplication (3 x 3 = 9)
    A = 2'b11; B = 2'b11; opcode = 3'b101; #10;

    // Division (valid case)
    A = 2'b10; B = 2'b01; opcode = 3'b110; #10;

    // Division by zero (error case)
    A = 2'b10; B = 2'b00; opcode = 3'b110; #10;

    $finish;
end
```

```
endmodule
```

5.3 CLA RTL Testbench Code

This testbench is used to check whether the CLA-based ALU performs all operations correctly for different input values. It creates a waveform file that can be viewed in GTKWave for output verification.

CLA Testbench

```
‘timescale 1ns/1ps

module tb_alu;

reg [1:0] A, B;
reg [2:0] opcode;
wire [3:0] result;
wire [1:0] remainder;
wire valid;
wire error;

alu_cla uut (
    .A(A),
    .B(B),
    .opcode(opcode),
    .result(result),
    .valid(valid),
    .error(error),
    .remainder(remainder)
);

initial begin
    $dumpfile("dump_cla.vcd");
    $dumpvars(0, tb_alu);

    A = 2'b11; B = 2'b11; opcode = 3'b000; #10;
    A = 2'b10; B = 2'b01; opcode = 3'b001; #10;
    A = 2'b11; B = 2'b01; opcode = 3'b010; #10;
    A = 2'b10; B = 2'b01; opcode = 3'b011; #10;
    A = 2'b11; B = 2'b01; opcode = 3'b100; #10;
    A = 2'b11; B = 2'b11; opcode = 3'b101; #10;
    A = 2'b11; B = 2'b10; opcode = 3'b110; #10;
    A = 2'b11; B = 2'b00; opcode = 3'b110; #10;

    $finish;
end

endmodule
```

5.3.1 Gate-Level Schematic of RCA and CLA

After synthesis, the Verilog design is converted into a gate-level schematic. This schematic shows how the circuit is built using logic gates and connections instead of code. It helps in understanding the real hardware structure generated from the HDL design.

The gate-level schematic of the **Ripple Carry Adder (RCA)** as in Figure 2 uses basic gates such as AND, OR, and XOR gates. In this design, the carry moves from the Least Significant Bit (LSB) stage to the Most Significant Bit (MSB) stage one by one. Because each stage waits for the previous carry, the circuit is simple but slower.

The gate-level schematic of the **Carry Lookahead Adder (CLA)** as in Figure 3 also uses AND, OR, and XOR gates, but it includes extra Generate and Propagate logic. This logic calculates carries in advance instead of waiting step by step. Due to this parallel carry generation, the CLA design is faster than the RCA.

So, RCA gives a simple and smaller design, while CLA gives better speed with slightly higher hardware complexity.

5.3.2 RCA vs CLA Comparison

Ripple Carry Adder (RCA) and Carry Lookahead Adder (CLA) are two important adder designs used in digital systems. Both perform binary addition, but their method of carry generation is quite different. Because of the difference, their speed, hardware usage, and complexity also change.

In RCA, the carry moves stage by stage from the Least Significant Bit (LSB) to the Most Significant Bit (MSB). Each full adder waits for the carry from the previous stage. This makes the design simple, but the total delay increases as the number of bits increases.

In CLA, the carry is calculated in advance using the Generate (G) and Propagate (P) logic. It erases the long waiting chain and gives quicker results, especially for larger bit-width adders. From the comparison, RCA is a good option when simplicity and low hardware cost are important. CLA is a better option when speed is the main requirement. Therefore, modern processors often prefer CLA or advanced fast adders for better performance.

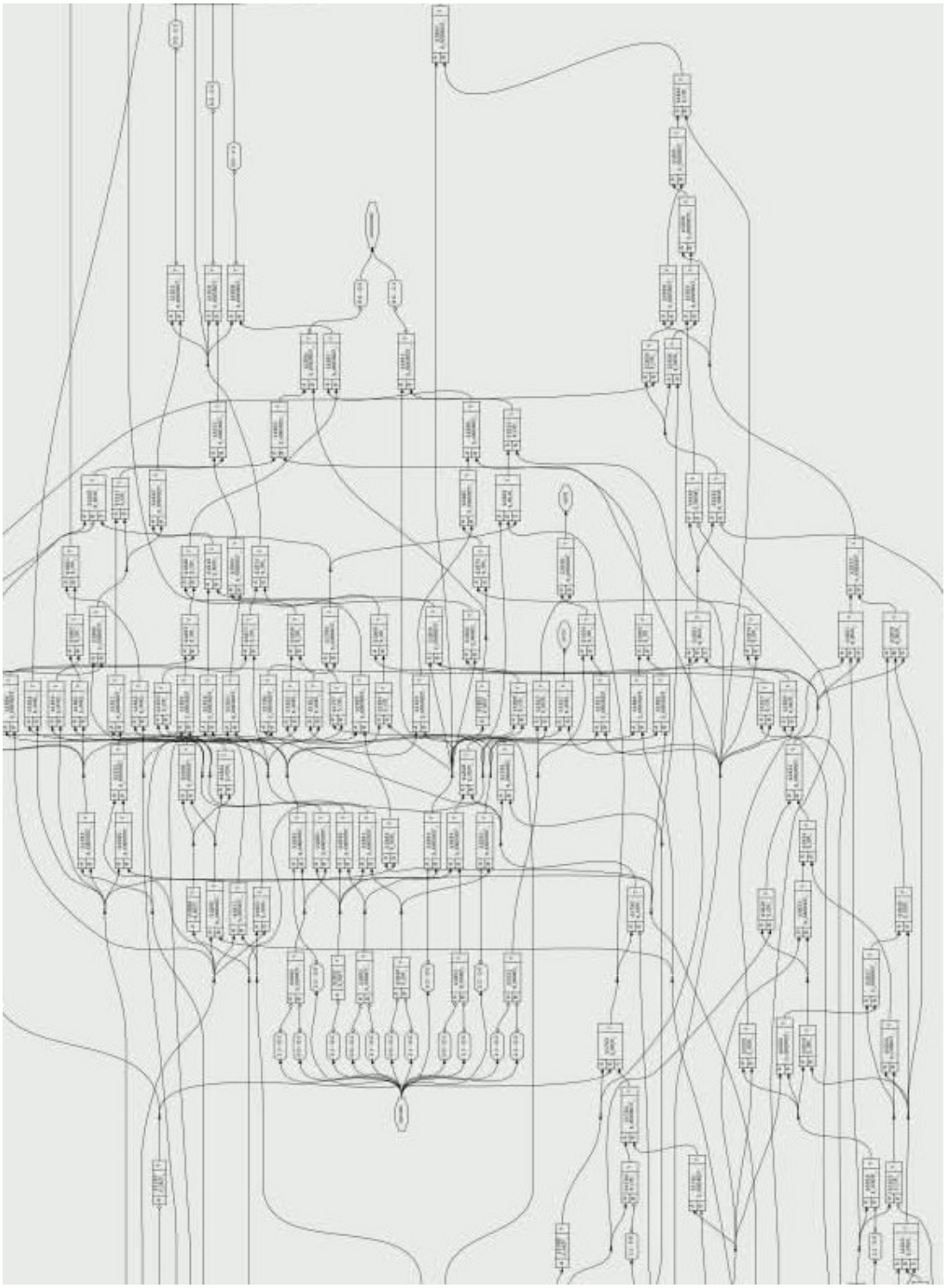

Figure 5.1: Gate-level schematic of RCA

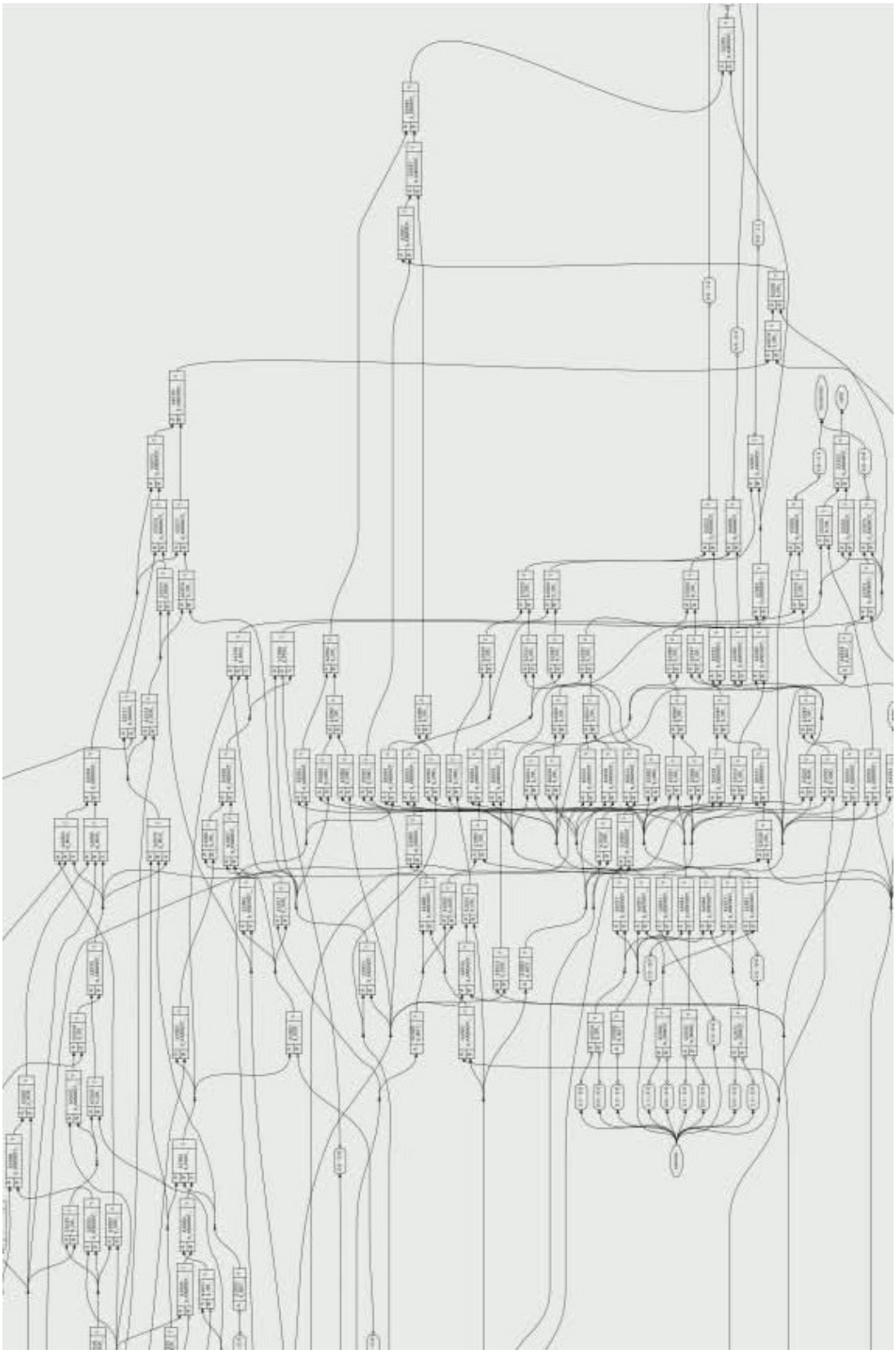

Figure 5.2: Gate-level schematic of CLA

Table 5.1: Comparison between Ripple Carry Adder and Carry Lookahead Adder

Parameter	**RCA**	**CLA**
Carry Method	Sequential carry ripple	Parallel carry lookahead
Speed	Slower	Faster
Delay Growth	Increases linearly with bits	Much lower delay
Hardware Complexity	Simple design	More complex design
Gate Count	Fewer gates/cells	More gates/cells
Area Required	Smaller area	Larger area
Power Usage	Usually lower	Slightly higher
Best Use Case	Small and low-cost circuits	High-speed processors and ALUs
Scalability	Poor for large bit sizes	Better for larger adders
Design Effort	Easy to implement	More difficult to design

Chapter 6 Physical Design Flow

After logic synthesis, the design moves to the physical design stage. At this stage, the gate-level netlist is converted into an actual chip layout that can be fabricated on silicon. Physical design focuses on arranging cells, connecting wires, managing power, and verifying the final layout.

The first step is **Floorplanning**. In this step, the chip area is decided and the locations of input/output pins, standard cells, and major blocks are planned. A good floorplan helps improve performance and routing efficiency.

The next step is **Power Planning**. Here, the power network for VDD and GND is created so that all cells receive proper power supply. This step is important for stable chip operation.

After that, **Placement** is performed. During placement, all standard cells are arranged inside the chip area in an optimized manner. The goal is to reduce wire length and improve timing.

Then comes **Clock Tree Synthesis (CTS)**. In this step, the clock signal is distributed to all required parts of the circuit with balanced delay. This ensures proper synchronization of sequential elements.

The next stage is **Routing**. Metal wires are created to connect all placed cells according to the circuit netlist .Routing completes the electrical connections of the design.

After routing, the layout is checked using **Magic VLSI**. It is used for design rule checking (DRC) and layout verification to ensure the chip follows fabrication rules.

Then the final layout is exported as a **GDSII file**, which is the standard file format used for chip fabrication.

Finally, the completed chip layout is viewed using **KLayout**. It helps visualise the final design and inspect the physical structure of the chip.

Thus, physical design transforms the synthesized logic into a manufacturable chip layout ready for fabrication.

6.1 Floorplanning

Floor planning is the first major step in physical design. It is like drawing a map before building a city. Before placing houses, roads, and parks, we first decide where everything should go. Likewise, before building the chip, we plan where all the important parts of the circuit will being placed.

A chip has limited space, so every block must be arranged carefully. During floor planning, the total chip area is decided, and the shape of the design is selected. Then, important blocks, standard cells, and input/output pins are assigned suitable positions.

The main goal of floor planning is to make later steps easier and quicker. If connected blocks are kept close to each other, the wires become shorter. Shorter wires reduce delay, save power, and improve performance.

For example, if two blocks need to exchange signals many times, placing them far away would require long wires. This wastes space and slows down the circuit. So floorplanning places related blocks near each other.

Another important task is keeping enough space for routing. Later, metal wires must pass between blocks. If everything is packed too tightly, routing becomes difficult.

Floorplanning decides where the power and ground connections can be distributed. This helps every part of the chip receive a stable power supply.

A good floorplan gives:

- Better speed
- Shorter wires
- Lower power usage
- Easier routing
- Better chip area usage

So, floorplanning is like making a smart blueprint of the chip before actual construction begins.

6.2 Power Planning

Power planning is the step where we decide how electricity will reach every part of the chip. It is like planning water pipes in a city. Just as every house needs water, every block inside the chip needs power to work properly.

A digital chip mainly uses two important power lines:

- **VDD** – Positive power supply
- **GND** – Ground connection

During power planning, metal paths are created to carry VDD and GND across the whole chip. These paths are called the Power Distribution Network (PDN).

The main goal is to make sure that every standard cell and logic block gets enough power. If power doesn't reach properly, the circuit becomes slow, unstable, or stops working.

Power planning helps reducing the voltage drop. If the path is too thin or too long, some energy is lost before reaching the cell. So, wider and stronger power lines are used where required.

Another benefit is noise reduction. A well-designed power network keeps the voltage stable even when many parts of the chip switch at the same time.

A good power plan gives:

- Stable chip operation
- Proper power to all cells
- Lower voltage drop
- Better reliability
- Improved performance

So, power planning is like building strong roads for electricity so the entire chip can run smoothly and safely.

6.3 Placement

Placement is the step where all standard cells are arranged inside the chip area after floorplanning and power planning. It is like arranging desks inside a classroom after deciding the room size. Every desk must fit properly, and enough space must remain for movement.

In digital design, standard cells are small brick-type boxes such as logic gates, adders, multiplexers, and flip-flops. During placement, these cells are positioned inside the floor-planned area in the best possible locations.

The main goal of placement is to keep connected cells close to each other. If related cells are near, the wires become shorter. Shorter wires help reduce delay and improve circuit speed.

Placement also avoids congestion. Congestion means too many cells or wires in one small area. If cells are packed too tightly, routing becomes difficult later.

Another goal is balancing performance and area. The tool tries to use space efficiently while also maintaining good timing.

For example, if an adder block and its output logic are placed far apart, the signal must travel as long distance. This increases the delay. So the placement tool arranges important connected cells closer together.

A good placement gives:

- Shorter wire length
- Better speed
- Easier routing
- Less congestion
- Efficient use of chip area

So, placement is just like arranging all parts of the chip in the right places before connecting them with wires.

6.4 Clock Tree Synthesis (OpenROAD)

Clock Tree Synthesis (CTS) in OpenROAD is the step where the clock signal is distributed to all sequential elements of the chip in a balanced and efficient manner. It is like one school bell sending the same sound to every classroom at the correct time.

In digital circuits, components such as flip-flops and registers depend on the clock signal. The clock tells them when to store data and when to update the outputs. If the clock reaches different parts at different times, the circuit may produce wrong hallucinating results.

OpenROAD automatically builds a network of wires and buffers to carry the clock signal from the source point to all important cells. This network is the clock tree.

The main purpose of CTS is to reduce **clock skew**. It is the difference in clock arrival time at different points of the chip. OpenROAD tries to make the clock reach all important cells as evenly as possible.

OpenROAD inserts buffers where needed. Buffers strengthen the clock signal and help it travel longer distances without delay or signal loss.

For example, if one flip-flop gets the clock early and another gets it late, data may be not transferred precisely. CTS in OpenROAD solves this by balancing the clock paths.

Advantages of Clock Tree Synthesis in OpenROAD:

- Balanced clock distribution
- Reduced clock skew
- Better timing performance
- Improved reliability
- Automatic buffer insertion

So, Clock Tree Synthesis in OpenROAD is like making equal roads for the clock signal so every part of the chip works together simultaneously and correctly.

6.5 Routing (OpenROAD)

Routing in OpenROAD is the step where all placed cells are connected using metal wires. It is like building roads between houses in a city after all houses have been placed in the correct locations.

After placement, the logic gates and other cells are inside the chip area, but they are not electrically connected yet. Routing creates the paths that allow signals and power to travel from one cell to another.

OpenROAD finds the best paths for these connections. It uses different metal layers available in chip manufacturing, like using bridges or flyovers when roads cross each other.

The main goal of routing is to connect every required pin without causing congestion, short circuits, or unnecessary delay. The tool makes sure to avoid overlapping of wires and tries to keep paths short and clean.

Routing is usually done in two parts:

- **Global Routing** – Plans the general path of wires
- **Detailed Routing** – Creates the exact final wire connections

For example, if one gate needs to send data to another gate, routing builds the metal path between them. If many signals need to cross, OpenROAD uses higher metal layers to avoid collisions.

A good routing stage gives:

- Complete signal connections
- Shorter wire length
- Lower delay
- Less congestion
- Reliable chip operation

So, routing in OpenROAD is like drawing smart roads inside the chip so signals can travel safely and quickly from one place to another.

6.6 Signoff Checks (Magic VLSI and LVS)

Sign-off checks are final verification steps performed before sending the chip for fabrication. It is like checking the homework carefully before submitting it. We make sure the design is correct, safe, and ready to go for manufacturing.

In this stage, tools such as **Magic VLSI** and **LVS** are used to verify the final chip layout. These checks helps ensure that the physical layout matches the intended circuit design.

The first important check is **DRC (Design Rule Check)**. This check is to make sure the layout follows fabrication rules such as proper wire width, spacing between metal lines, and correct layer usage.

For example, if two wires are placed too close to each other, they may short-circuit during manufacturing. DRC detects such mistakes and reports them.

The second important check is **LVS (Layout Versus Schematic)**. This check compares the final layout with the original circuit netlist or schematic.

If the layout connections and components match the intended design, the LVS check passes but if something is missing, extra, or wrongly connected, LVS report an error.

For example, if one gate is connected to the wrong wire in the layout, LVS will detect that mismatch.

The main goals of sign-off checks are:

- Detect design rule violations
- Verify layout matches schematic
- Prevent short circuits and open connections
- Improve manufacturing reliability
- Prepare the chip for fabrication

If all checks pass, the design is considered ready for tape-out and fabrication.

So, signoff checks using Magic VLSI and LVS are the final quality test of the chip before the manufacturing process.

6.7 GDS Generator, Final Chip Layout and KLayout Visualization

After all design steps and verification checks are complete, the final chip layout is generated as **GDSII file**. This file is the final blueprint of the chip and is used for semiconductor fabrication.

GDSII file contains complete physical information, such as cell placement, routing wires, vias, metal layers, and geometric shapes. Fabrication machines read this file to manufacture the silicon chip.

To view and inspect this layout, the GDSII file is opened in **KLayout**. KLayout is a very powerful layout viewer used in VLSI design. It helps engineers zoom into the chip, check layers, observe routing paths, and visually inspect the final physical structure of chip.

KLayout is useful because it allows us to understand how the digital design has been converted into a real chip layout. It also helps compare different designs - RCA and CLA.

The layouts below represent the final physical implementation of both adders used in the ALU design.

The RCA layout is based on sequential carry propagation. The structure is simpler and uses fewer logic resources, making the layout easier to implement. The CLA layout

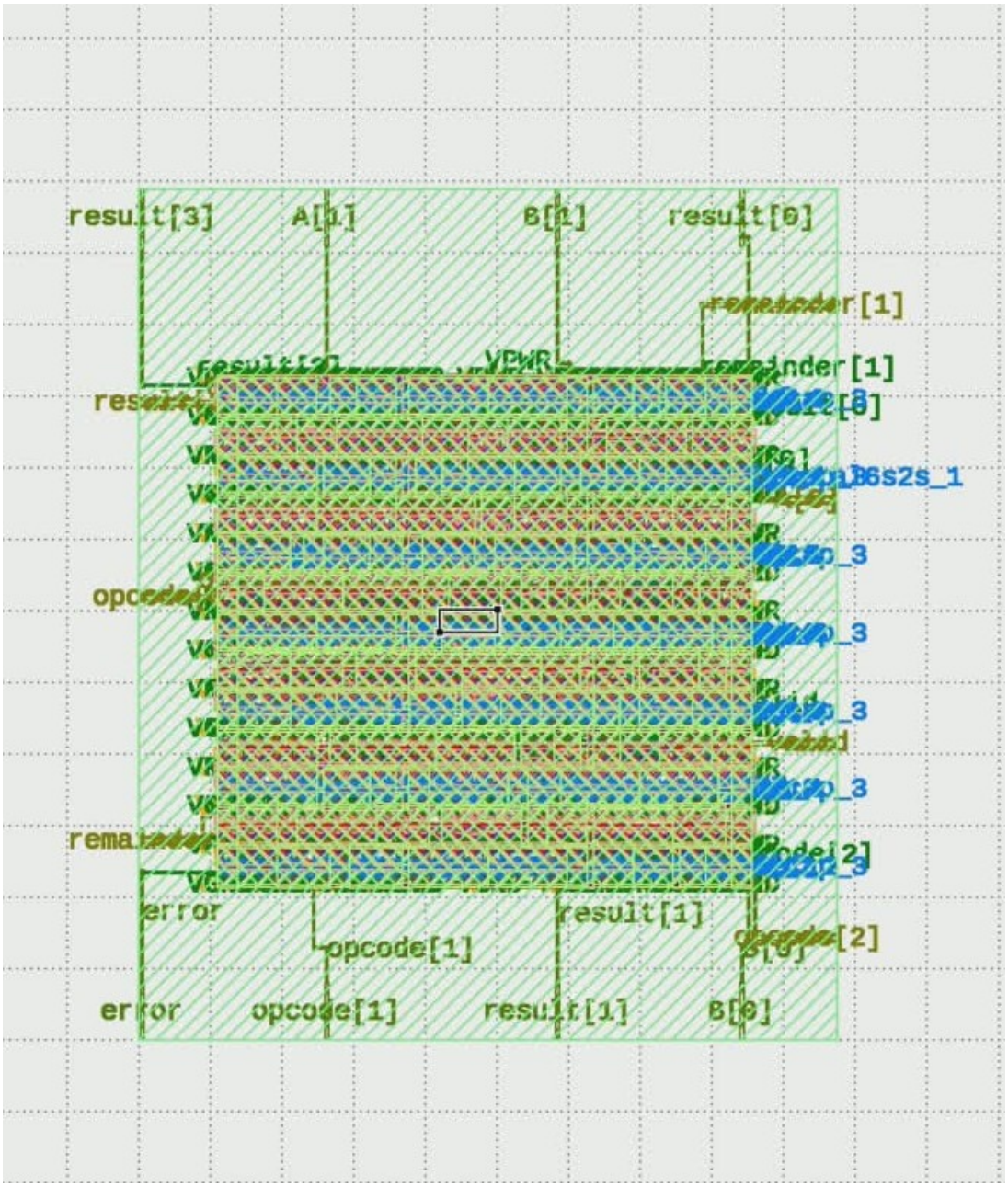

Figure 6.1: Final chip layout of RCA viewed in KLayout

includes additional Generate and Propagate logic for quicker carry calculation. Because of this additional logic, the layout appears dense and slightly more complex than RCA.

Thus, KLayout helps in visualising the completed chip; the final layouts confirm that both RCA and CLA designs successfully pass the physical design flow and are all set for fabrication.

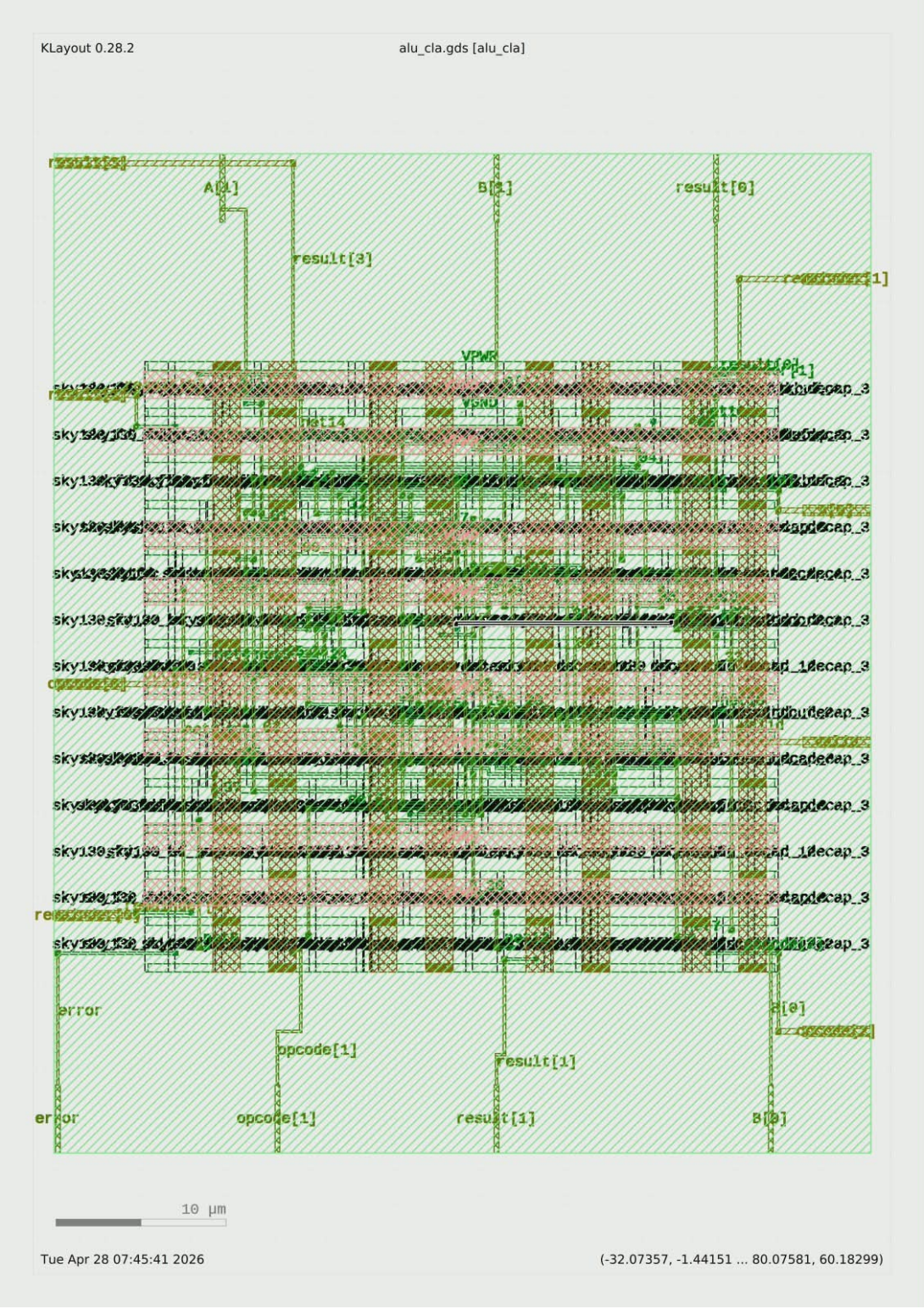

Figure 6.2: Final chip layout of CLA viewed in KLayout

Conclusive Remarks

This project helped us understand how a small Arithmetic Logic Unit (ALU) can be designed using Verilog HDL. Step by step, we created a 2-bit ALU that can perform different operations such as addition, subtraction, AND, OR, XOR, multiplication, and division. We also added quotient, remainder, and error handling for division. After testing the design in simulation, we confirmed that the ALU was giving the correct outputs for different input values.

We also learned that the way we design one small part of the circuit can change the overall performance. For addition, we compared two methods: Ripple Carry Adder (RCA) and Carry Lookahead Adder (CLA). RCA is easy to build and uses less hardware, but it takes more time because the carry moves one step at a time. CLA is faster because it calculates carry signals in advance, but it needs more hardware. This teaches us an important lesson in engineering: making something faster often means making it more complex.

Most importantly, this project was not only about making an ALU. It was about learning how real digital systems are designed, tested, and improved. We used tools like Icarus Verilog, GTKWave, and Yosys to see how code becomes hardware. Even though this was a small 2-bit design, the same ideas are used in bigger processors and modern computers. So, this project is a strong first step toward understanding future technologies and VLSI design.

References

[1] M. Morris Mano, *Digital Design*, Pearson Education.

[2] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, Pearson Education.

[3] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Pearson Education.

[4] Yosys Open Synthesis Suite, *Official Documentation*. Available: `https://yosyshq.net/yosys/`

[5] OpenLane Documentation, *The OpenROAD Project*. Available: `https://openlane.readthedocs.io/`

[6] OpenROAD Tool, *The OpenROAD Project*. Available: `https://theopenroadproject.org/`

[7] SkyWater Technology Foundry, *SkyWater 130nm PDK Documentation*. Available: `https://skywater-pdk.readthedocs.io/`

[8] Icarus Verilog, *Documentation*. Available: `http://iverilog.icarus.com/`

[9] GTKWave, *Documentation*. Available: `http://gtkwave.sourceforge.net/`

[10] A. Ghazy and M. Shalan, "OpenLANE: The Open-Source Digital ASIC Flow," Efabless.

[11] M. Shalan and T. Edwards, "Building OpenLANE: A 130nm OpenROAD-based Flow," Efabless.

[12] A. B. Kahng et al., "OpenROAD: Toward a Self-Driving Layout Tool Chain," University of California, San Diego.

[13] D. S. Lopera et al., "Using Open-Source EDA Tools in Industrial Flow."